

PhysicalTraining 최종 보고서

AI 맞춤 루틴과 기록 기반 성장을 지원하는 Android 헬스 트레이닝 앱

1. 프로젝트 개요 및 기획 의도

PhysicalTraining은 사용자의 신체 정보, 운동 경력, 운동 목적을 바탕으로 맞춤형 운동 루틴을 추천하고, 세트별 중량·반복 횟수·휴식시간을 함께 관리할 수 있도록 제작한 Android 기반 헬스 트레이닝 앱입니다. 단순히 운동 이름을 기록하는 앱이 아니라, 사용자가 오늘 수행할 루틴을 확인하고, 각 세트를 체크하며, 휴식 타이머와 운동 기록까지 한 흐름 안에서 관리할 수 있도록 설계하였습니다.

이 앱을 기획하게 된 계기는 개인적인 운동 경험에서 시작되었습니다. 웨이트 트레이닝을 지속하면서 단순히 운동을 하는 것보다 기록을 남기고 이를 바탕으로 점진적 과부하를 적용하는 것이 중요하다는 점을 느꼈습니다. 그러나 실제 운동 과정에서는 매번 루틴을 구성하고, 적절한 중량을 계산하고, 세트별 휴식시간을 맞추며, 운동 기록을 따로 정리하는 과정이 번거롭게 느껴졌습니다.

최근에는 건강 관리와 근력 유지를 목적으로 웨이트 트레이닝을 시작하는 사용자도 늘어나고 있습니다. 하지만 초보자는 어떤 운동을 해야 하는지, 어느 정도 중량과 반복 횟수로 수행해야 하는지 판단하기 어렵습니다. 반대로 이미 운동을 하고 있는 사용자도 매번 운동 계획을 세부적으로 조정하고 기록하는 과정에서 불편함을 느낄 수 있습니다.

PhysicalTraining의 핵심 가치는 개인화, 실행 편의성, 기록 기반 성장 관리입니다. 사용자는 성별, 나이, 체중, 운동 경력, 운동 목적을 입력하면 자신에게 맞는 운동 프로그램과 세트별 중량을 추천받을 수 있습니다. 운동 중에는 체크리스트와 자동 휴식 타이머를 통해 수행 흐름을 유지할 수 있으며, 완료한 운동 기록은 저장되어 성장 그래프에 반영됩니다.

2. 시스템 아키텍처 및 기술 스택 선정 이유

앱은 UI 계층, ViewModel 상태 관리 계층, Repository 계층, 로컬/원격 데이터 저장 계층, AI 추천 계층으로 나누어 구성하였습니다. 사용자는 Compose 화면에서 루틴과 운동 세트를 조작하고, ViewModel은 화면 상태와 비즈니스 로직을 관리합니다. Repository는 Room DB와 Firebase Firestore를 연결하며, TensorFlow Lite 모델은 사용자 프로필을 입력값으로 받아 기준 중량을 예측합니다.

기술 스택	사용 위치	선정 이유
Kotlin	Android 앱의 주요 구현 언어	Android 공식 권장 언어이며 Compose, Coroutine, ViewModel 등 Android Jetpack 생태계와 잘 맞기 때문에 선택하였습니다.
Jetpack Compose	전체 UI 화면 구현	선언형 UI 방식으로 화면 상태와 UI 를 연결하기 쉽고, 루틴/체크리스트/그래프처럼 상태 변화가 많은 화면을 간결하게 구성할 수 있습니다.
Room Database	루틴, 세트, 사용자 프로필, 운동 기록 로컬 저장	운동 앱은 네트워크가 불안정해도 기록을 유지해야 하므로 로컬 DB 가 필요하였습니다. Room 은 SQLite 를 안전하게 다룰 수 있고 Flow 와 함께 실시간 UI 갱신이 가능합니다.
Firebase Auth	회원가입/로그인 인증	사용자별 백업 데이터를 분리하기 위해 로그인 기반 uid 가 필요하였고, Firebase Auth 는 Android 에서 빠르게 인증 기능을 붙일 수 있습니다.
Firebase Firestore	운동 기록 원격 백업/복원	운동 기록을 기기 외부에 보관하고 재설치 후 복원하기 위해 사용하였습니다. 문서 기반 구조라 사용자별 기록 리스트 저장에 적합합니다.
TensorFlow Lite	온디바이스 AI 기준 중량 예측	헬스장치럼 네트워크가 불안정할 수 있는 환경에서도 AI 추천이

기술 스택	사용 위치	선정 이유
		동작하도록 서버 API 대신 앱 내부 모델을 선택하였습니다.
Coil / Assets	운동 이미지 표시	운동 종목을 텍스트만으로 보여주면 이해가 어렵기 때문에 assets 에 운동 이미지와 설명 JSON 을 저장하고 화면에서 함께 표시하였습니다.

3. 핵심 기능 구현 세부 사항

본 프로젝트의 핵심 기능은 AI 기반 맞춤 루틴 및 중량 처방, 체크리스트 중심 운동 수행 관리와 자동 백업, 기록 기반 성장 그래프입니다. 세 기능은 각각 "운동 계획 생성", "운동 수행", "운동 결과 확인"에 해당하며 앱의 전체 사용 흐름을 구성합니다.

3.1 AI 맞춤 루틴 및 운동 추가 자동 처방

```
RoutineWeightCalculator.kt  lines 151-186

151 fun calculateAddedExercisePrescription(
152     aiResult: FloatArray,
153     exerciseName: String,
154     goal: String
155 ): AddedExercisePrescription {
156     val bench1RM = aiResult.getOrNull(0)?.takeIf { it > 0f } ?: 0f
157     val squat1RM = aiResult.getOrNull(1)?.takeIf { it > 0f } ?: 0f
158     val deadlift1RM = aiResult.getOrNull(2)?.takeIf { it > 0f } ?: 0f
159     val ohp1RM = bench1RM * 0.65f
160
161     val exerciseType = classifyAddedExercise(exerciseName)
162     val preset = goalPreset(goal, exerciseType)
163     val sets = preset.sets
164     val reps = preset.reps
165     val restTime = preset.restTime
166
167     val weight = if (exerciseType == AddedExerciseType.BODYWEIGHT || exerciseType == AddedExerciseType.CORE) {
168         0f
169     } else {
170         val exerciseOneRepMax = estimateAddedExerciseOneRepMax(
171             exerciseName = exerciseName,
172             bench1RM = bench1RM,
173             squat1RM = squat1RM,
174             deadlift1RM = deadlift1RM,
175             ohp1RM = ohp1RM
176         )
177         val targetRepWeight = exerciseOneRepMax / (1f + reps / 30f)
178         roundedPositiveWeight(targetRepWeight)
179     }
180
181     return AddedExercisePrescription(
182         sets = sets,
183         reps = reps,
184         weight = weight,
185         restTime = restTime
186     )
}
```

AI 모델은 사용자의 신체 정보로 벤치프레스, 스쿼트, 데드리프트 기준 중량을 예측합니다. 이후 운동 이름을 기준으로 복합 운동, 고립 운동, 맨몸 운동, 코어 운동을 분류하고 사용자의 운동 목표에 맞는 세트 수, 반복 횟수, 휴식시간을 결정합니다. 중량 운동은 3 대 운동 기준값에 운동별 비율을 적용해 추천 중량을 계산하고, 풀업·딥스·푸쉬업·코어 운동은 중량 대신 횟수 중심으로 처방합니다.

3.2 운동 체크리스트, 휴식 타이머, 자동 백업 연동

```
WorkoutViewModel.kt  lines 363-405

363 fun toggleSet(routineId: String, exercise: String, setIndex: Int) {
364     var toggleSEtEntity: WorkoutSetEntity? = null
365
366
367     _routines.update { currentList ->
368         currentList.map { routine ->
369             if (routine.id == routineId) {
370                 val newMap = routine.exercises.toMutableMap()
371                 val currentSets = newMap[exercise]?.toMutableList() ?: return@map routine
372
373                 if (setIndex in currentSets.indices) {
374                     val updateSet = currentSets[setIndex].copy(
375                         isChecked = !currentSets[setIndex].isChecked
376                     )
377                     currentSets[setIndex] = updateSet
378                     newMap[exercise] = currentSets
379
380                     if (updateSet.isChecked) {
381                         startRestTimer(updateSet.restTime)
382                     }
383
384                     toggleSEtEntity = WorkoutSetEntity(
385                         setId = updateSet.setId,
386                         routineId = routineId,
387                         exerciseName = exercise,
388                         weight = updateSet.weight,
389                         reps = updateSet.reps,
390                         isChecked = updateSet.isChecked,
391                         restTime = updateSet.restTime
392                     )
393                 }
394                 routine.copy(exercises = newMap)
395             } else routine
396         }
397     }
398     toggleSEtEntity?.let { entity ->
399         viewModelScope.launch {
400             repository.updateSet(entity)
401             syncWorkoutHistoryForToggledSet(routineId, entity)
402             backupWorkoutHistorySnapshotToFirebase()
403             checkAndCreateNextWeekRoutine(routineId)
404         }
405     }
```

사용자가 세트 완료 체크박스를 누르면 체크 상태가 즉시 변경되고 Room DB 에 저장됩니다. 체크된 세트는 운동 히스토리로 저장되며, 체크 해제 시에는 해당 기록을 삭제합니다. 완료 체크 시에는 세트에 저장된 휴식시간으로 타이머가 자동 시작되고, 이후 현재 완료 기록을 Firebase 에 자동 백업합니다. 백업 성공은 조용히 처리하고, 네트워크 오류 등 실패 상황에서만 스낵바를 표시해 운동 흐름을 방해하지 않도록 설계하였습니다.

3.3 성장 그래프 기록 집계 및 시각화

```
GraphScreen.kt  lines 67-85

67     val filteredHistory = workoutHistory.filter { history ->
68         history.isCompleted && history.exerciseName.contains(selectedExercise)
69     }
70
71     val weeklyGraphData = filteredHistory
72         .groupBy { history ->
73             SimpleDateFormat("yyyy-ww", Locale.getDefault())
74                 .format(Date(history.completedAt))
75         }
76         .map { (week, histories) ->
77             WeeklyGraphPoint(
78                 weekLabel = week,
79                 maxWeight = histories.maxOf { it.weight },
80                 volume = histories.sumOf {
81                     (it.weight * it.reps).toDouble()
82                 }.toFloat()
83             )
84         }
85         .sortedBy { it.weekLabel }
```

성장 그래프는 완료된 운동 기록만 대상으로 합니다. 사용자가 선택한 운동명과 일치하는 기록을 필터링한 뒤, 완료 날짜를 주차 단위로 묶습니다. 각 주차별 최고 중량은 해당 주의 가장 무거운 완료 세트로 계산하고, 총 볼륨은 무게와 반복 횟수를 곱한 값을 모두 더해 계산합니다. 이를 통해 사용자는 단순 완료 여부뿐 아니라 실제 중량 상승과 훈련량 변화를 확인할 수 있습니다.

4. UI/UX 디자인 콘셉트

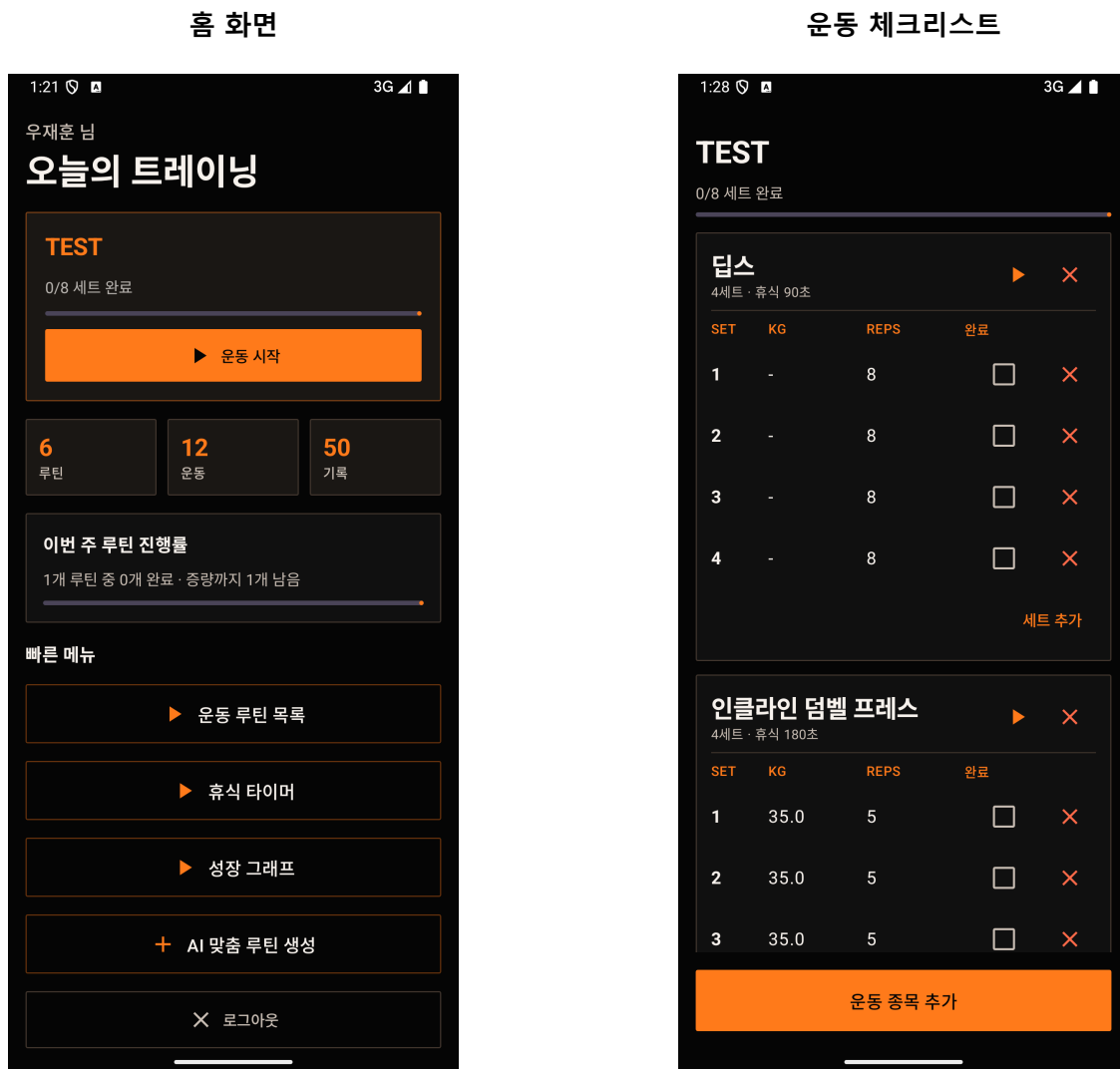
본 앱의 UI 디자인은 어두운 다크 콘셉트를 중심으로 구성하였습니다. 전체 배경은 검정 계열로 통일하고, 버튼과 강조 요소에는 주황색 포인트 컬러를 사용하여 사용자가 주요 기능을 쉽게 인식할 수 있도록 하였습니다. 운동 이미지가 어두운 배경 위에서도 잘 보이도록 이미지 배경을 투명하게 처리하고, 운동 종목 추가 화면에서는 상단 이미지 영역을 고정하여 사용자가 리스트를 스크롤해도 선택한 운동 이미지를 계속 확인할 수 있도록 하였습니다.

UX 측면에서는 실제 운동 중 느낄 수 있는 불편함을 줄이는 데 초점을 맞추었습니다. 사용자는 루틴 화면에서 오늘 수행할 운동을 확인하고, 체크리스트에서 세트 완료 여부를 기록하며, 완료 시 자동으로 휴식 타이머와 운동 기록 저장 흐름으로 이어집니다. 또한 운동 추가 화면에서는 검색, 부위별 분류, 운동 설명, 이미지 미리보기를 제공하여 새로운 운동을 빠르게 찾고 추가할 수 있도록 구성하였습니다.

결과적으로 PhysicalTraining 은 단순히 보기 좋은 화면보다 실제 운동 중 빠르게 확인하고 조작할 수 있는 실용적인 UX 를 우선하였습니다. 홈 화면에는 오늘의 루틴과 이번 주 루틴 진행률을 함께 배치하여 사용자가 현재 상태와 다음 목표를 한눈에 파악할 수 있도록 하였습니다.

5. 주요 화면 캡처

아래 화면은 최종 코드 기준으로 에뮬레이터에서 새로 캡처한 주요 기능 화면입니다.



오늘의 트레이닝, 루틴/운동/기록 요약, 이번 주 루틴 진행률을 한 화면에서 확인합니다.

세트별 중량, 반복 횟수, 완료 여부를 관리하고 완료 시 휴식 타이머와 자동 백업 흐름으로 이어집니다.

운동 종목 추가

1:41

3G

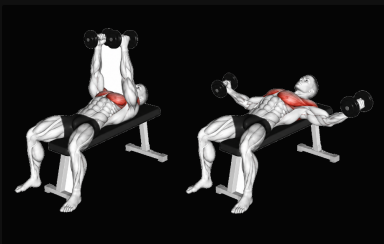
✕

운동 종목 추가

TEST

덤벨 플라이

가슴



가슴을 벌리고 모으는 고립 운동입니다. 팔꿈치를 살짝 굽힌 상태로 가슴 근육의 늘어남과 수축을 느낍니다.

운동 검색

가슴

덤벨 플라이

가슴

가슴을 벌리고 모으는 고립 운동입니다. 팔꿈치를 살짝 굽힌 상태로 가슴 근육의 늘어남과 수축을 느낍니다.

덤스

가슴

프로필 목표와 AI 3대 운동값으로 세트, 횟수, 무게, 휴식시간이 자동 설정됩니다.

추가

운동 이미지, 부위, 설명, 검색 기능을 제공하고 AI 기준값으로 세트/횟수/무게/휴식시간을 자동 설정합니다.

성장 그래프



완료된 운동 기록을 바탕으로 주차별 최고 중량과 총 볼륨 변화를 시각화합니다.

AI 맞춤 루틴 추천 화면

3:12 3G

AI 운동 추천 입력

우재훈 님,

신체 정보와 목표를 입력하시면, AI가 가장 최적화된 주 5일 루틴과 무게를 추천해 드립니다.

성별

☒ 남성 ☐ 여성

몸무게 (kg)

75.0

나이 (세)

25

운동 경력

중급자

운동 목적

스트레스

AI 추천 적응 프로그램

nSuns 5/3/1

i

AI 맞춤 루틴 생성 및 추천받기

신체 정보, 운동 경력, 운동 목적을 기반으로 AI 맞춤 루틴과 기준 중량을 생성합니다.

6. 트러블 슈팅 및 문제 해결

AI 모델 정확도 한계 문제

초기에는 사용자의 신체 정보, 운동 경력, 운동 목적을 바탕으로 모든 운동 종목의 추천 중량을 AI 모델이 직접 예측하도록 구성하고자 하였습니다. 그러나 개인별 신체 조건, 운동 경력, 수행 자세, 운동 숙련도, 기구 차이 등에

따라 적정 중량이 크게 달라지기 때문에 모든 운동 종목에 대해 정확한 추천 중량을 예측하는 모델을 만드는 데 한계가 있었습니다.

이를 해결하기 위해 AI 모델이 모든 운동의 중량을 직접 예측하는 방식에서 벗어나, 먼저 벤치프레스, 스쿼트, 데드리프트와 같은 주요 3 대 운동의 기준 중량을 추정하도록 구조를 변경하였습니다. 이후 각 운동 종목별로 기준 운동과 비율을 설정하고, 3 대 운동 추정값에 운동별 파생 공식을 적용하여 다른 운동들의 추천 중량을 계산하도록 개선하였습니다.

이 방식은 AI 모델이 모든 운동을 직접 예측하는 것보다 안정적이었습니다. 모델은 사용자의 기본 근력 수준을 추정하는 역할을 담당하고, 세부 운동 중량은 운동 생리학적 관계와 루틴 템플릿에 따른 비율 계산으로 보완하였습니다.

데이터 백업 및 복원 오류 문제

두 번째로 어려웠던 부분은 운동 기록 백업과 복원 기능이었습니다. 앱에는 사용자가 수행한 운동 기록을 안전하게 보관하기 위해 Firebase 를 통한 백업 기능을 추가하였습니다. 하지만 초기 구현 단계에서는 백업 버튼을 눌러도 실제로 어느 단계에서 문제가 발생하는지 바로 확인하기 어려웠습니다.

이를 해결하기 위해 백업과 복원 과정의 각 단계마다 로그를 출력하도록 수정하였습니다. 현재 로그인된 사용자 정보 확인, 백업할 데이터 개수 확인, Firestore 업로드 요청, 업로드 성공 여부, 복원 시 불러온 데이터 개수 등을 단계별로 Logcat 에 표시하도록 하였습니다.

이후에는 사용자가 별도로 백업 버튼을 누르지 않아도 운동 체크리스트에서 세트를 완료하거나 해제할 때마다 현재 완료 기록이 자동으로 Firebase 에 동기화되도록 개선하였습니다. 네트워크 문제로 백업이 실패한 경우에만 앱 화면에 안내 스낵바를 표시하여 운동 흐름을 방해하지 않도록 하였습니다.

7. 프로젝트 후기 및 향후 발전 방향

이번 프로젝트를 진행하면서 가장 크게 느낀 점은 AI 도구의 발전으로 앱 기능을 구현하는 진입 장벽이 낮아졌다는 것입니다. 운동 루틴 데이터를 입력하거나 JSON 파일을 정리하는 반복 작업은 AI 의 도움을 받아 효율적으로 처리할 수 있었습니다. 이를 통해 개발자는 단순 데이터 입력보다 앱의 전체 구조와 사용자 경험을 고민하는 데 더 많은 시간을 사용할 수 있었습니다.

하지만 동시에 앱 개발에서 UX 의 중요성이 더욱 커졌다고 생각하였습니다. 기능을 구현하는 것 자체는 가능했지만, 사용자가 실제로 편하게 사용할 수 있는 앱을 만드는 것은 별개의 문제였습니다. 운동 중 화면을

확인하는 방식, 세트를 체크하는 흐름, 타이머 사용 방식 등에서 작은 불편함이 실제 사용성에 큰 영향을 준다는 점을 배웠습니다.

시간 관계상 구현하지 못해 아쉬웠던 기능은 운동 자세 분석 기능입니다. 사용자가 운동하는 사진이나 짧은 영상을 입력하면 앱이 자세를 분석하여 문제점을 알려주고 코칭해주는 기능을 추가하고 싶었습니다. 향후에는 운동 기록과 루틴 추천 기능을 넘어, 실제 운동 자세까지 분석할 수 있는 종합적인 헬스 트레이닝 앱으로 발전시키고 싶습니다.

8. 프로젝트 요구사항 만족여부 체크리스트

요구사항	만족 여부	구현 내용
화면 전환 4 개 이상	충족	로그인/회원가입, 프로필 입력, 홈, 루틴 목록, 체크리스트, 운동 추가, 타이머, 성장 그래프를 구성하였습니다.
리스트 화면 1 개 이상	충족	루틴 목록, 체크리스트, 운동 종목 추가 화면에서 LazyColumn 기반 리스트를 사용하였습니다.
이미지 포함	충족	운동 종목별 PNG/GIF 이미지를 assets 에 저장하고 운동 추가 화면에서 표시하였습니다.
데이터베이스 사용	충족	Room DB 를 사용하여 루틴, 운동 세트, 사용자 프로필, 운동 히스토리를 저장하였습니다.
Firebase / 외부 API / AI / MAP 중 1 개 이상	충족	Firebase Auth/Firestore 와 TensorFlow Lite 온디바이스 AI 모델을 함께 사용하였습니다.